FlakeScope: Ranking Flaky Test Failures

Lena Brandt
University of Nowhere
lena.brandt@uni-nowhere.edu

***Abstract*—Flaky tests pass and fail on the same code. FlakeScope ranks the failures of a build by the chance that each failure is flaky. In our previous work [1], we built the data set that this paper uses.**

I. Introduction

Flaky tests pass and fail on the same code. Such tests waste the time of developers, hide real faults, and erode the trust in continuous integration. This paper presents FlakeScope, a tool that ranks the tests of a project by the chance that a failure is flaky. FlakeScope reads the history of the test runs and the changes of each commit.

Our evaluation covers ten open source projects with 4,210 test failures. FlakeScope places the flaky failure first in 71% of the cases, compared with 38% for a rerun of the failed tests alone.

II. Background

A test is flaky when it passes and fails on the same version of the code. Common causes are asynchronous waits, shared state between tests, and the order in which the tests run. A rerun of each failure detects some flaky tests. However, the rerun costs time on every build.

III. Approach

FlakeScope collects three signals for each failure. The first signal is the history of the test, such as the fraction of past failures that a rerun fixed. The second signal is the distance between the failing test and the changed code. The third signal is the order of the tests in the failing run. A gradient boosted model combines the three signals into one score.

IV. Evaluation

We collected the build logs of ten projects over two years. Two annotators labeled each failure as flaky or real and agreed on 92% of the failures before a discussion of the rest. FlakeScope ranks 2,988 of 4,210 failures correctly at the first position. The history signal contributes the most, followed by the distance signal.

A. Projects

The ten projects span build tools, web frameworks, databases, and libraries for data processing. Each project runs its tests on every commit in a public continuous integration service and keeps the logs of at least two years. The projects hold between 1,200 and 18,000 tests and between 40 and 900 test failures per year. Seven projects run their tests in parallel, which makes failures that depend on the order of the tests more common.

B. Accuracy of the Ranking

For each failed build, we sort the failures by the score of FlakeScope and record the position of the first flaky failure. FlakeScope places the flaky failure first in 71% of the builds and within the first three positions in 89% of the builds. A rerun of the failed tests places the flaky failure first in 38% of the builds, since a rerun misses flaky tests that fail again by chance. The accuracy stays above 65% in every project, and the project with the most parallel tests reaches the highest accuracy.

C. Contribution of the Signals

We remove each signal in turn and repeat the ranking. Without the history signal, the accuracy drops to 52%. Without the distance signal, it drops to 63%. Without the order signal, it drops to 68%. The three signals therefore complement each other, and the history of a test carries the most information about its flakiness.

D. Cost

FlakeScope scores the failures of a build in less than two seconds on a laptop. The model trains in four minutes on the history of one project. A rerun of every failed test, in contrast, adds a median of eleven minutes to a failed build in the ten projects.

V. Discussion

Developers can use the ranking in two ways. First, the ranking can order the failures in the report of a build, such that the failures that need attention come first. Second, a project can rerun only the failures that rank low, which saves the time of the reruns that a flaky failure would waste. Both uses keep a developer in the loop, since a ranking never hides a failure.

The ranking also helps the maintainers of a project to find the tests that fail most often for reasons outside the code under test. These tests are candidates for a fix of their waits, their shared state, or their order.

VI. Threats to Validity

The labels of the failures come from two annotators who read the logs and the code. Their agreement of 92% indicates a reliable labeling, but some failures remain hard to judge from a log alone. The ten projects are open source projects with public logs, and industrial projects may show other causes of flakiness. The model of FlakeScope trains on the history of each project, therefore a new project needs a few weeks of history before the ranking becomes accurate.

VII. Related Work

Prior work studies the causes of flaky tests and detects order-dependent tests. FlakeScope complements these detectors with a ranking that a developer can read after each failed build.

VIII. Conclusion

FlakeScope ranks the failures of a build by the chance that each failure is flaky. A tool paper with the full evaluation and the data set follows. FlakeScope is available at https://github.com/flaky_lab/flaky-tool.

Acknowledgments

We thank the University of Nowhere for its support.

References

[1] L. Brandt, "A data set of flaky test failures," in *Proc. MSR*, 2025.